

SHARELOOP · SWP391 · NHÓM 1

LỘ TRÌNH TRIỂN KHAI

Tuần 4 → tuần 9: từ các quyết định đã chốt tới khi code backend và các mốc tới khi kết thúc

Việc cần làm · Cấu trúc thư mục · Công cụ · Kiến thức · Lưu ảnh khi deploy

Dành cho: 4 thành viên Backend (Java 21 · Spring Boot 4.1.1) và 1 thành viên Frontend

Đi kèm: SRS ShareLoop v10 (cập nhật 29/09) và Luồng hoạt động & ERD v10 (ERD đã chốt, có cột chuẩn)

Ngày: 29/09/2026 — đầu tuần 4 (cập nhật cột chuẩn, Spring Boot 4.1.1)

MỤC LỤC

1. Mục đích của tài liệu.....	3
1.1. Thế nào là "sẵn sàng code".....	3
1.2. Phân vai trong nhóm.....	3
2. Tổng quan các bước.....	5
3. Chi tiết từng bước.....	6
3.1. Bước 1 — Các quyết định đã chốt và việc tương ứng trong code.....	6
3.2. Bước 2 và 3 — Môi trường, công cụ và sơ đồ.....	7
3.3. Bước 4 — Migration từ ERD đã chốt.....	7
3.4. Bước 5 — API contract.....	9
3.5. Bước 6 — Dựng skeleton backend.....	9
3.6. Bước 7 — Quy ước làm việc.....	13
3.7. Bước 8 — Thử nghiệm kỹ thuật (spike).....	14
3.8. Bước 9 — Backlog theo tuần.....	14
4. Cấu trúc thư mục và cách 4 người code song song.....	16
4.1. Năm nguyên tắc.....	16
4.2. Cấu trúc repo.....	16
4.3. Cấu trúc thư mục backend.....	16
4.4. Bên trong một module.....	18
4.5. Module nào cần khi nào.....	19
4.6. Module — người sở hữu — dữ liệu được ghi.....	19
4.7. Gọi chéo module: service cửa ngõ.....	20
4.8. Bảng users và items: entity "lát cắt".....	21
4.9. Migration Flyway khi 4 người cùng sửa schema.....	23
4.10. Thư mục resources và test.....	23
4.11. CODEOWNERS và những file "nóng".....	23
5. Công cụ phát triển: IDE và trợ lý AI.....	26
5.1. Cài đặt chung (cả 4 BE).....	26
5.2. Thiết lập từng IDE.....	26
5.3. Giữ mã đồng nhất giữa các IDE.....	27
5.4. Trợ lý AI: một bộ hướng dẫn cho mọi công cụ.....	27
6. Kiến thức cần học.....	30
6.1. Phần cả nhóm backend đều phải nắm.....	30
6.2. Phần học thêm theo module.....	30
6.3. Cách học hiệu quả trong thời gian ngắn.....	31
7. Những gì còn phải tìm hiểu.....	32
8. Phương án lưu hình ảnh khi deploy (C12).....	33
8.1. Ước lượng dung lượng.....	33
8.2. So sánh các phương án.....	33
8.3. Thiết kế StorageService.....	34
8.4. Các bước triển khai lưu ảnh.....	35
9. Lịch làm việc từ tuần 4 đến tuần 9.....	36
9.1. Tuần 4 theo từng ngày.....	36
9.2. Các tuần tiếp theo.....	36
10. Checklist sẵn sàng code.....	38
11. Những lỗi hay gặp khi bắt đầu và cách tránh.....	39
Nguồn tham khảo.....	40

Nếu mở bằng Microsoft Word mà mục lục chưa hiện số trang: bấm Ctrl+A rồi F9, chọn "Update entire table".

1. Mục đích của tài liệu

SRS v10 trả lời câu hỏi "hệ thống làm gì", tài liệu Luồng hoạt động & ERD trả lời "dữ liệu nằm ở đâu". Tài liệu này trả lời câu hỏi **"4 bạn backend bắt đầu code thế nào, để trong thư mục nào, dùng công cụ gì, học gì"**. Dự án đang ở **đầu tuần 4** và **kết thúc ở tuần 9**. Sáng 28/9 nhóm đã chốt các điểm nhóm 1 (chương 15 SRS) và **chốt ERD 17 bảng**, nên phần chuẩn bị còn lại gói trong **3 ngày 28/9 – 30/9**; code tính năng từ thứ Năm 1/10.

So với bản 27/09, bản này thay đổi bốn điểm: backend dùng **Java 21 + Spring Boot 4.1.1** (nhánh đang được hỗ trợ, trùng môi trường bài thi thực hành môn Java); bảng chính có **7 cột chuẩn** theo yêu cầu giảng viên (C34); thêm **chương 4 — cấu trúc thư mục** để 4 người code song song ít đụng nhau; thêm **chương 5 — công cụ** cho nhóm dùng nhiều IDE (IntelliJ IDEA, Antigravity, VS Code) và nhiều trợ lý AI (Claude Code, Codex, Gemini...); nơi lưu ảnh khi deploy (C12) **lùi tới khi web chạy ổn định**.

1.1. Thế nào là "sẵn sàng code"

Mỗi điều kiện là đầu ra của một bước ở chương 3. **Điều kiện 1–7 phải xong trước tối thứ Năm 1/10**; điều kiện 8 xong trước khi module cần tới (tuần 4–5).

- 1. **Đã xong 28/9**: các điểm nhóm 1 và ERD 17 bảng đã chốt, ghi vào biên bản `docs/decisions/` và SRS đã sửa theo.
- 2. Cả 4 bạn backend đã cài JDK 21, IDE và Docker theo chương 5; mở được project và chạy test trên máy mình.
- 3. `V1__init.sql` (17 bảng) và các migration dữ liệu khởi tạo `V2–V4` chạy sạch trên database trống.
- 4. Skeleton backend theo **đúng cây thư mục ở chương 4**: đủ package của 18 module, CODEOWNERS, AGENTS.md; chạy bằng một lệnh; CI build, test và kiểm định dạng mã đều xanh.
- 5. API contract cho các module của tuần 4–5 đã viết trong `docs/api/`; FE nối hoặc mock theo đúng contract.
- 6. Các service "cửa ngõ" giữa module (mục 4.7) đã có chữ ký hàm và bản rỗng được merge, để người gọi code trước mà không phải chờ.
- 7. Quy ước làm việc (nhánh, commit, PR, review, định dạng lỗi API, định dạng mã, dùng trợ lý AI) đã ghi trong README.
- 8. Các thử nghiệm kỹ thuật rủi ro cao (thanh toán, email, khoá dòng ví, AI) đã chạy thử thành công ít nhất một lần.

1.2. Phân vai trong nhóm

Vai	GitHub	Package phụ trách (chương 4)	Nhiệm vụ
BE1 — nhóm trưởng	arisdo-29	<code>auth, user, wallet, listingfee</code> ; lớp nền <code>common, config</code> ; <code>integration/payment, integration/mail</code>	Đăng ký, OTP email, đăng nhập, hồ sơ, đổi số; ví, nạp VNPAY; phí đăng bài (giữ / trừ / nhà), phí sửa, gia hạn, đẩy bài; dựng skeleton.
BE2	Nguyentri2531	<code>catalog, media, item, moderation, search</code> ; <code>integration/ai, integration/storage</code>	Danh mục, thuộc tính, khu vực; ảnh; bài đăng; kiểm duyệt nội dung (lexicon, regex, AI sàng lọc); tìm kiếm; AI trợ lý.

Vai	GitHub	Package phụ trách (chương 4)	Nghịệp vụ
BE3	dwargon73-sketch	<code>request, chat, notification</code>	Request và state machine, chat (gọi bộ lọc của BE2), chốt lịch, trao liên lạc, giao nhận, thông báo; quy ước job nền.
BE4	kopslnbtram2110	<code>setting, review, report, reputation, admin, audit</code>	Cấu hình, lexicon, checklist; hàng chờ duyệt; khiếu nại; TrustStars, điểm, hạng; khoá tài khoản, báo cáo tài chính; nhật ký.
FE	My-Mieu	<code>frontend/</code>	Màn hình theo screen flow, mock theo API contract, nối API khi có.

Mỗi package có đúng một người chịu trách nhiệm

Người sở hữu package là người viết chính và là người duyệt mọi PR chạm vào package đó (tự động qua CODEOWNERS, mục 4.11). Nhóm đối phân công tùy ý, chỉ cần sửa bảng này và file CODEOWNERS cùng lúc.

2. Tổng quan các bước

Bước	Việc	Ai làm	Đầu ra	Thời gian
1	Chốt quyết định nhóm 1 và ERD 17 bảng	Cả nhóm; BE1 chủ trì	Biên bản, SRS cập nhật	Xong 28/9
2	Cài môi trường và công cụ (chương 5)	Từng người	Mở được project, chạy test	T2 28/9
3	Cập nhật screen flow theo các mức phí đã chốt	FE + BE1	Screen flow mới	28/9 – 2/10
4	Viết migration từ ERD đã chốt (có cột chuẩn C34)	BE1 (V1), BE4 (V2, V4), BE2 (V3)	Migration chạy sạch	T4 30/9
5	API contract phần tài khoản, bài đăng, ví	Mỗi BE phần module mình; FE review	Contract sprint tuần 4–5	28/9 – 30/9
6	Dựng skeleton theo cây thư mục chương 4 (Hướng dẫn BE1)	BE1 (cập cùng một BE khác)	Repo chạy được, CI xanh	29/9 – 1/10
7	Thống nhất quy ước làm việc	BE1 soạn, cả nhóm duyệt	Mục "Quy ước" trong README	T3 29/9
8	Thử nghiệm kỹ thuật (spike)	Theo module, giới hạn thời gian	Đoạn mã chạy được + ghi chú	30/9 – 9/10, xen kẽ code
9	Backlog sprint tuần 4, chia việc	BE1	Issue trên GitHub Projects	T4 30/9
→	Bắt đầu code tính năng	4 BE + FE	Mốc tuần 4: đăng ký → đăng nhập → tạo bài	từ T5 1/10

Các bước 2, 4, 5, 6, 7 chạy song song trong ba ngày đầu tuần. Việc học (chương 6) diễn ra xuyên suốt, mỗi người học trước phần cần cho việc của tuần đó.

3. Chi tiết từng bước

3.1. Bước 1 — Các quyết định đã chốt và việc tương ứng trong code

Buổi chốt sáng 28/9 đã khép gần hết chương 15 của SRS. Bảng dưới chuyển từng quyết định thành việc cụ thể trong mã, để người phụ trách biết mình phải làm gì.

Mã	Quyết định	Việc trong code	Ai
C01, C02	Mọi bài 5 Credit / 30 ngày; giữ khi gửi, trừ khi duyệt lần đầu	<code>PostFeeService.holdPostFee / chargeHeldFee / releaseHeldFee</code> ; khoá <code>post_fee</code>	BE1
C14, C14b	Sửa: miễn phí khi bị chặn hoặc chưa duyệt; 1 lần miễn phí sau duyệt; từ lần sau 5 Credit/lần	<code>GET /items/{id}/edit-quote</code> ; <code>holdEditFee</code> ; khoá <code>edit_fee = 5</code>	BE1 + BE2
C10	Bài đang hiển thị khi sửa thì ẩn tạm tới khi duyệt xong	<code>ItemService.submitEdit</code> đưa bài về <code>PENDING_REVIEW</code> ; Request Pending giữ nguyên	BE2
C16	Gia hạn 5 Credit / 30 ngày , không duyệt lại	<code>RenewService</code> ; khoá <code>renew_fee</code>	BE1
C15	Đẩy bài 5 Credit / 3 ngày , cộng dồn ≤ 14 ngày, ≤ 3 bài/trang	<code>BoostService</code> ; truy vấn tìm kiếm xếp bài đang đẩy lên trước	BE1 + BE2
C17, C18	AI trợ lý 2 Credit/lượt ; 5 lượt miễn phí/ngày khi đã nạp; chưa nạp 1 lượt thử	<code>AiAssistantService</code> kiểm hạn mức; khoá <code>ai_search_fee = 2</code>	BE2
C04	Vẫn chặn liên hệ trong chat	<code>ChatService</code> gọi <code>ContentModerationService</code> trước khi lưu tin	BE3 + BE2
C05	SĐT duy nhất trong tài khoản ACTIVE; đổi số bằng OTP email, 1 lần/30 ngày	Partial unique index trong V1; <code>UserService.changePhone</code>	BE1
C08	AI sàng lọc bài chỉ gắn cờ	<code>ItemScreeningService</code> chỉ ghi <code>ai_risk</code> , <code>ai_result</code> ; không đổi <code>status</code>	BE2
C09	Danh mục theo SRS 6.17; cấm động vật sống	Dữ liệu <code>V3__seed_catalog.sql</code> , cờ <code>is_restricted</code>	BE2
C11	VNPay sandbox; dự phòng cổng giả lập	<code>PaymentGatewayClient</code> hai bản cài đặt: <code>VnPayClient</code> , <code>MockPaymentClient</code>	BE1
C06	PostgreSQL managed có gói miễn phí hoặc cùng nhà cung cấp với nơi lưu ảnh	Chọn dịch vụ cụ thể ở spike S11, trước deploy thử tuần 7	BE4
C33	ERD 17 bảng đã chốt	<code>V1__init.sql</code> ; thay đổi sau đó chỉ bằng migration mới (mục 4.9)	BE1
C34	Bảng chính có 7 cột chuẩn; bảng danh mục 6 cột (ERD mục 7.11)	<code>BaseEntity</code> + JPA Auditing; <code>@SQLRestriction</code> xoá mềm; trigger <code>set_updated_at</code> trong V1	BE1
C13	Không cần hỏi lại giảng viên	Đóng	—

Còn mở

C07 nhà cung cấp LLM và **C31** trần ngân sách AI: chốt tuần 5 theo spike S5 (BE2). **C12** nơi lưu ảnh khi deploy: chốt **sau khi web chạy ổn định**, trước deploy thử tuần 7 (BE2, chương 8). **C26** thời điểm đối tác xem chứng từ: chốt trước khi làm xác thực hàng chính hãng ở tuần 8 (BE4 + nhóm). Mọi con số nghiệp vụ nằm trong `website_attributes`, nên đổi giá trị sau này không phải sửa mã.

3.2. Bước 2 và 3 — Môi trường, công cụ và sơ đồ

- **Môi trường:** mỗi người làm theo chương 5 trong buổi sáng 28/9. Người xong trước hỗ trợ người sau; cuối ngày cả 4 người chạy được `./mvnw verify`.
- **Use case:** bốn sơ đồ v10 đã có trong SRS chương 17. Nếu nhóm vẫn nộp file drawio thì sửa theo đúng các hình đó.
- **Screen flow:** popup phí 5 Credit và chính sách hoàn phí khi đăng bài; trước khi sửa hiện loại lượt (miễn phí / **5 Credit**); màn hình gia hạn (5 Credit / 30 ngày); màn hình đẩy bài (5 Credit / 3 ngày); trang AI trợ lý hiện lượt miễn phí còn lại và giá **2 Credit** khi hết lượt; nút chia sẻ thẻ bài trong chat; câu hỏi "đúng mô tả?" khi xác nhận nhận hàng; màn hình quản lý từ khoá cấm của Admin; bỏ bước OTP số điện thoại khỏi đăng ký.
- FE đối chiếu các trang đã làm trong `frontend/src/pages` với screen flow mới, liệt kê trang cần sửa. Ví dụ tiện ích `aiMatching.ts` thuộc tính năng AI ghép đôi đã bỏ từ v9.

3.3. Bước 4 — Migration từ ERD đã chốt

ERD không còn phải review: việc của bước này là chép đúng từ điển dữ liệu (chương 7 tài liệu ERD) thành SQL. Bốn file khởi tạo, mỗi file một người:

File	Nội dung	Người viết
<code>V1__init.sql</code>	Extension <code>unaccent</code> , <code>pg_trgm</code> ; hàm <code>f_unaccent</code> ; 17 bảng; ràng buộc và index ở mục 7.7 ERD	BE1
<code>V2__seed_config.sql</code>	Toàn bộ giá trị cấu hình ở mục 7.10 ERD (<code>attr_group = CONFIG</code>)	BE4
<code>V3__seed_catalog.sql</code>	Danh mục, thuộc tính động, khu vực (tỉnh → quận/huyện)	BE2
<code>V4__seed_lexicon_checklist.sql</code>	Từ khoá cấm khởi tạo, 10 mục checklist, lý do từ chối mẫu, tên miền email dùng một lần	BE4

Quy ước khi viết (giữ nguyên với mọi migration sau này):

- Tên bảng và cột snake_case chữ thường, bảng số nhiều (`users`, `items`, `credit_ledger`).
- Khoá chính `id BIGINT GENERATED ALWAYS AS IDENTITY`; khoá ngoại đặt tên `<bảng_số_ít>_id`.
- Cột chuẩn theo nhóm bảng (ERD mục 7.11): 5 bảng chính đủ `created_at`, `updated_at`, `created_by`, `updated_by`, `is_active`, `is_deleted`, `status`; 4 bảng danh mục, cấu hình có 6 cột (không `status`); bảng tài chính, nhật ký, mapping giữ gọn. `created_by`, `updated_by` không đặt khoá ngoại.
- Không xoá cứng bản ghi nghiệp vụ; ràng buộc duy nhất chỉ tính bản ghi chưa xoá (`WHERE NOT is_deleted`). Trigger `set_updated_at` gán cho mọi bảng có `updated_at`.
- Tiền và Credit dùng `BIGINT`, có `CHECK (credit_balance >= 0)`, `CHECK (held_credit >= 0)`.

- Enum lưu `VARCHAR + CHECK (status IN (...))`; dữ liệu linh hoạt (kết quả AI, bản chụp checklist) dùng `JSONB`.
- Ràng buộc nghiệp vụ nào đưa được xuống DB thì đưa: SĐT duy nhất trong tài khoản ACTIVE (partial index), một Request đang hoạt động cho mỗi cặp (item, receiver), `payment_order_id` duy nhất trong sổ Credit.

```
-- V1__init.sql (phần đầu)
CREATE EXTENSION IF NOT EXISTS unaccent;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- unaccent() không phải IMMUTABLE nên không dùng trực tiếp trong index; bọc lại:
CREATE OR REPLACE FUNCTION f_unaccent(text) RETURNS text
LANGUAGE sql IMMUTABLE PARALLEL SAFE STRICT
AS $$ SELECT public.unaccent('public.unaccent', $1) $$;

CREATE OR REPLACE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$ BEGIN NEW.updated_at := now(); RETURN NEW; END $$;

CREATE TABLE users (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email       VARCHAR(255) NOT NULL,
  phone      VARCHAR(15)  NOT NULL,
  password_hash VARCHAR(100) NOT NULL,
  credit_balance BIGINT NOT NULL DEFAULT 0 CHECK (credit_balance >= 0),
  held_credit  BIGINT NOT NULL DEFAULT 0 CHECK (held_credit >= 0),
  status      VARCHAR(20) NOT NULL DEFAULT 'PENDING_VERIFICATION'
              CHECK (status IN ('PENDING_VERIFICATION', 'ACTIVE', 'BANNED')),
  is_admin    BOOLEAN NOT NULL DEFAULT FALSE,
  -- ... các cột còn lại theo từ điển dữ liệu mục 7.3 ERD
  -- 6 cột chuẩn (ERD 7.11), cùng status ở trên là đủ 7
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by  BIGINT,
  updated_by  BIGINT,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  is_deleted  BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE UNIQUE INDEX ux_users_email ON users(email) WHERE NOT is_deleted;
CREATE UNIQUE INDEX ux_users_phone_active ON users(phone)
  WHERE status = 'ACTIVE' AND NOT is_deleted;
CREATE TRIGGER trg_users_updated_at BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- V2__seed_config.sql (ví dụ vài dòng)
INSERT INTO website_attributes
(attr_group, attr_key, attr_value, sort_order) VALUES
('CONFIG', 'post_fee',      '5', 0),  -- cột chuẩn lấy giá trị mặc định
('CONFIG', 'edit_fee',      '5', 0),
('CONFIG', 'renew_fee',     '5', 0),
('CONFIG', 'ai_search_fee', '2', 0);
```

3.4. Bước 5 — API contract

Contract là thứ FE và BE cùng dựa vào để làm song song. Viết trước khi code, mỗi module một file markdown trong `docs/api/` (ví dụ `docs/api/item.md`). Khi backend đã chạy, springdoc sinh Swagger từ mã; nhóm đối chiếu hai bản để phát hiện chỗ lệch.

Quy ước chung

Nội dung	Quy ước
Tiền tố	<code>/api/v1/...</code>
Tên tài nguyên	Danh từ số nhiều: <code>/items, /requests/{id}</code> ; API của Admin: <code>/api/v1/admin/...</code>
Hành động nghiệp vụ	Động từ con: <code>POST /requests/{id}/select, POST /items/{id}/boost</code>
Xác thực	Header <code>Authorization: Bearer <JWT></code>
Phân trang	<code>?page=0&size=20&sort=createdAt,desc</code> ; trả <code>{ items, page, size, totalElements }</code>
Thời gian	ISO-8601 UTC, ví dụ <code>2026-10-10T03:00:00Z</code> ; FE tự đổi sang giờ Việt Nam
Mã HTTP	200/201 thành công; 400 sai dữ liệu; 401 chưa đăng nhập; 403 không có quyền; 404 không thấy; 409 xung đột trạng thái (vd bài đã Reserved); 422 vi phạm business rule

Định dạng lỗi thống nhất

```
{
  "code": "ITEM_CONTAINS_CONTACT",
  "message": "Mô tả có số điện thoại. Vui lòng xóa để tiếp tục.",
  "field": "description",
  "details": { "match": "09xx xxx xxx" },
  "traceId": "a1b2c3"
}
```

`code` là hằng số trong enum mã lỗi **của từng module** (`ItemErrorCode`, `WalletErrorCode...`, tiền tố theo module), FE dựa vào `code` để hiển thị, không dựa vào `message`. Danh sách mã lỗi là một phần của contract.

Thứ tự viết contract

1. Auth (đăng ký, OTP, đăng nhập, đổi mật khẩu) — FE đã có trang, cần nổi sớm nhất.
2. Danh mục, khu vực (đọc), tải ảnh và bài đăng (tạo, xem, tìm kiếm).
3. Ví, nạp tiền, phí đăng bài, báo giá sửa bài (`edit-quote`).
4. Request và chat.
5. Admin: hàng chờ duyệt, cấu hình, lexicon.

3.5. Bước 6 — Dựng skeleton backend

3.5.1. Tạo project

Dùng Spring Initializr (start.spring.io hoặc trình tạo project có sẵn trong IntelliJ / VS Code): **Maven**, **Java 21**, **Spring Boot 4.1.1**, group `com.shareloop`, artifact `backend`, package `com.shareloop`, packaging `jar`. Sau khi tạo, kiểm tra thẻ `parent` của `pom.xml` đúng **4.1.1** và commit ngay, để cả 4 người build ra cùng một bộ thư viện:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
</parent>
<properties>
  <java.version>21</java.version>
</properties>
```

Dependency	Để làm gì
Spring Web	REST API (Spring MVC) — artifact <code>spring-boot-starter-webmvc</code> (Spring Boot 4 đổi tên từ <code>-web</code>)
Spring Security	Xác thực, phân quyền
OAuth2 Resource Server	Tạo và kiểm JWT bằng <code>JwtEncoder</code> / <code>JwtDecoder</code> có sẵn — <code>spring-boot-starter-security-oauth2-resource-server</code> (tên mới ở Spring Boot 4)
Spring Data JPA	Truy cập PostgreSQL
PostgreSQL Driver	JDBC driver
Flyway Migration	Quản lý schema — <code>spring-boot-starter-flyway</code> + <code>flyway-database-postgresql</code> (Spring Boot 4 bắt buộc có starter)
Validation	<code>@Valid</code> , <code>@NotBlank</code> ... cho DTO
Java Mail Sender	Gửi OTP và thông báo
Spring Boot Actuator	Endpoint <code>/actuator/health</code> cho deploy
Lombok	Bớt code getter/constructor (IDE phải bật annotation processing)
Testcontainers	Test trên PostgreSQL thật — <code>spring-boot-testcontainers</code> , Testcontainers 2.x: <code>testcontainers-postgresql</code> , <code>testcontainers-junit-jupiter</code>
Thêm tay trong pom.xml	<code>springdoc-openapi-starter-webmvc-ui</code> dòng 3.x (dòng dành cho Spring Boot 4) để có Swagger UI; test starter theo công nghệ: <code>spring-boot-starter-webmvc-test</code> , <code>spring-boot-starter-data-jpa-test</code> ; plugin Spotless (mục 5.3); AWS SDK v2 S3 khi làm bản lưu ảnh S3

Vì sao Spring Boot 4.1.1, và lưu ý khi đọc hướng dẫn cũ

Nhánh 3.5 đã hết hỗ trợ miễn phí từ 30/06/2026; **4.1.1** (20/08/2026) là bản vá mới nhất của nhánh đang được hỗ trợ, và bài thi thực hành môn Java cũng dùng bản này. **Không đổi phiên bản** khi IDE, Dependabot hay trợ lý AI gợi ý; đổi bản Spring Boot phải cả nhóm đồng ý.

Phần lớn hướng dẫn trên mạng vẫn viết cho Spring Boot 3. Những chỗ khác khi làm với Spring Boot 4:

- Starter đổi tên, tách nhỏ: `spring-boot-starter-webmvc` thay `-web`; Flyway cần `spring-boot-starter-flyway`; test có starter riêng cho từng công nghệ (`-webmvc-test`, `-data-jpa-test`). Không dùng starter "classic" (chỉ để chuyển dự án cũ).
- Jackson 3: `ObjectMapper/JsonMapper` nằm ở package `tools.jackson.databind`; annotation như `@JsonProperty` vẫn ở `com.fasterxml.jackson.annotation`.
- Test: dùng `@MockitoBean`; `@MockBean` đã bị gỡ khỏi Spring Boot 4.
- Testcontainers 2: artifact có tiền tố `testcontainers-` (vd `testcontainers-postgresql`); dùng lớp `org.testcontainers.postgresql.PostgreSQLContainer`, lớp cũ trong `org.testcontainers.containers` đã deprecated.

- Spring Security 7: cấu hình bằng bean `SecurityFilterChain`; hướng dẫn cũ dùng `WebSecurityConfigurerAdapter` không dùng được.

3.5.2. Cấu trúc thư mục

Skeleton tạo **đủ cây thư mục của chương 4 ngay từ đầu**, kể cả package chưa có mã (đặt một file `package-info.java` ghi người sở hữu). Như vậy ngay đầu code không ai phải tạo package mới, và mỗi người chỉ làm trong thư mục của mình.

3.5.3. Môi trường dev bằng Docker Compose

Mỗi người chỉ cần Docker Desktop và chạy `docker compose up -d` trong thư mục `backend/` là có PostgreSQL, hộp thư giả để xem email OTP, và MinIO để thử lưu ảnh kiểu S3.

```
# backend/docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shareloop
      POSTGRES_USER: shareloop
      POSTGRES_PASSWORD: shareloop
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
  mail:
    # SMTP giả: xem email tại http://localhost:8025
    image: axllent/mailpit
    ports: ["1025:1025", "8025:8025"]
  minio:
    # tùy chọn, thư'StorageService kiểu S3
    image: minio/minio
    command: server /data --console-address ":9001"
    ports: ["9000:9000", "9001:9001"]
volumes:
  pgdata:

# src/main/resources/application-dev.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/shareloop
    username: shareloop
    password: shareloop
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate # schema do Flyway quản lý
    properties.hibernate.jdbc.time_zone: UTC
  flyway:
    locations: classpath:db/migration,classpath:db/dev-data
    out-of-order: true # chỉ ở dev, xem mục 4.9
  mail: { host: localhost, port: 1025 }
app:
  jwt:
    secret: ${JWT_SECRET:dev-only-secret-change-me-32-bytes-min}
    ttl-minutes: 120
  storage:
    type: local # local | s3
  ai:
    enabled: false # bật khi đã có API key
```

```
timeout-seconds: 20
```

Ba profile: **dev** (máy cá nhân), **test** (Testcontainers), **prod** (đọc mọi bí mật từ biến môi trường). **Không commit** API key, mật khẩu SMTP, secret JWT; file **.env** nằm trong **.gitignore**, chỉ commit **.env.example**.

3.5.4. Những lớp nền cần có ngay (BE1)

- **PageResponse**, **ErrorResponse** và định dạng lỗi ở mục 3.4; API thành công trả thẳng DTO, không bọc.
- Interface **ErrorCode** + **BusinessException** + **GlobalExceptionHandler** (**@RestControllerAdvice**) biến mọi lỗi thành JSON thống nhất. Mỗi module tự khai báo enum **<Module>ErrorCode implements ErrorCode**.
- **BaseEntity** có **id** và 6 cột chuẩn (**createdAt**, **updatedAt**, **createdBy**, **updatedBy**, **isActive**, **isDeleted**), điền bằng JPA Auditing với **AuditorAware<Long>** đọc id từ JWT; hàm **markDeleted()**; lớp **SliceAudit** cho entity lát cắt (ERD mục 7.11). Bean **Clock** để test chỉnh được thời gian cho job.
- **SecurityConfig**: mở **/api/v1/auth/****, **/actuator/health**, Swagger; **/api/v1/admin/**** yêu cầu quyền ADMIN; còn lại cần JWT. Nhờ quy ước URL này, thêm API mới **không phải sửa SecurityConfig**.
- **@CurrentUserId** để controller lấy id người đang đăng nhập mà không đụng tới Spring Security.
- Một endpoint mẫu đủ các tầng (**GET /api/v1/categories** trong package **catalog**) kèm một unit test và một integration test, để mọi người copy theo đúng khuôn.
- Các service "cửa ngõ" ở mục 4.7 với chữ ký hàm và thân rỗng, merge trong tuần 4.

3.5.5. CI trên GitHub Actions

```
# .github/workflows/backend.yml
# Build + kiểm định dạng + test backend. Chạy trên mọi PR để dùng làm
# "required check"; PR không đóng vào backend/ thì bỏ qua các bước nâng.
name: backend
on:
  pull_request:
  push:
    branches: [develop, main]
jobs:
  backend-build:
    name: backend-build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: PR có sửa backend không
        id: changes
        env:
          EVENT: ${github.event_name}
          BASE: ${github.event.pull_request.base.sha}
        run: |
          if [ "$EVENT" != "pull_request" ]; then
            echo "run=true" >> "$GITHUB_OUTPUT"
          elif git diff --name-only "$BASE...HEAD" \
            | grep -qE '^(backend/|\.github/workflows/backend\.yaml)'; then
            echo "run=true" >> "$GITHUB_OUTPUT"
```

```

    else
      echo "run=false" >> "$GITHUB_OUTPUT"
    fi
  - uses: actions/setup-java@v4
    if: steps.changes.outputs.run == 'true'
    with:
      distribution: temurin
      java-version: "21"
      cache: maven
  - name: Kiểm tra dinh dạng (Spotless)
    if: steps.changes.outputs.run == 'true'
    working-directory: backend
    run: ./mvnw -B spotless:check
  - name: Build và test (ca Testcontainers)
    if: steps.changes.outputs.run == 'true'
    working-directory: backend
    run: ./mvnw -B verify

```

Workflow chạy trên mọi PR để dùng làm check bắt buộc; PR không đụng **backend/** thì bỏ qua các bước nặng và vẫn xanh, nên PR của FE không bị kẹt. Repo có thêm workflow **no-ai-attribution** chặn dòng ghi công AI (Hướng dẫn BE1, bước B2).

Bật luật bảo vệ nhánh **develop** và **main**: bắt buộc PR, bắt buộc CI xanh, bắt buộc review của code owner; mã trong **wallet/**, **listingfee/** và **request/** cần 2 người duyệt.

3.6. Bước 7 — Quy ước làm việc

Nội dung	Quy ước
Nhánh	main (bản nộp / demo), develop (tích hợp), feature/ <module>-<mô-tả> (vd feature/wallet-topup-vnpay), fix/ <module>-<mô-tả>
Commit	Dạng feat(item): thêm API tạo bài, fix(wallet): ..., test(request): ..., docs: ...; phạm vi trong ngoặc là tên package
Pull request	Nhỏ (dưới ~400 dòng), mỗi PR chủ yếu trong một module; dùng template trong .github/ , ghi rõ FR/BR liên quan
Review	Code owner của package được hỏi tự động; mã chạm tiền, trạng thái, bảo mật cần 2 người
Định dạng mã	Spotless quyết định, không phải IDE: chạy ./mvnw spotless:apply trước khi commit (mục 5.3)
Test	Mỗi business rule có ít nhất một test; tên test mô tả hành vi, vd holdFee_whenBalanceInsufficient_throws
DTO	Không trả entity ra API; request/response là record ; kiểm tra đầu vào bằng annotation Validation
Cấu hình	Mọi con số nghiệp vụ (phí, hạn mức, mốc thời gian) đọc qua ConfigService từ website_attributes , không viết cứng
Mã do AI sinh	Được dùng theo mục 5.4; người mở PR phải hiểu và giải thích được từng dòng; commit dưới tên người làm

3.7. Bước 8 — Thử nghiệm kỹ thuật (spike)

Spike là đoạn mã nhỏ, làm nhanh, để chắc một kỹ thuật rủi ro chạy được **trước khi** đưa vào kế hoạch. Mỗi spike giới hạn thời gian; hết giờ mà chưa xong thì báo nhóm để đổi hướng sớm. Mã spike để trong nhánh `spike/<tên>`, không merge thẳng vào `develop`.

Spike	Mục tiêu "xong"	Ai	Giới hạn
S1 — VNPay sandbox	Tạo URL thanh toán, trả về, nhận IPN, kiểm chữ ký HMAC-SHA512 thành công ở localhost (dùng ngrok hoặc tương tự để nhận IPN).	BE1	1,5 ngày
S2 — Email OTP	Gửi OTP qua Mailpit ở dev và qua Gmail App Password thật một lần.	BE1	0,5 ngày
S3 — Khoá dòng ví	Test 2 luồng đồng thời cùng giữ phí trên một tài khoản chỉ đủ tiền một lần: đúng một luồng thành công. Dùng entity lát cắt <code>WalletAccount</code> (mục 4.8).	BE1	1 ngày
S4 — Chuẩn hoá tiếng Việt + regex	Hàm chuẩn hoá và regex số điện thoại bắt được 20 mẫu lách luật, không bắt nhầm 10 mẫu hợp lệ.	BE2	1 ngày
S5 — Gọi LLM trả JSON	Gọi 2–3 nhà cung cấp với cùng 20 mẫu bài đăng, nhận JSON đúng schema, đo thời gian và ước chi phí. Căn cứ chốt C07, C31.	BE2	1,5 ngày
S6 — Tìm kiếm không dấu	Truy vấn "ao khoac" ra bài "Áo khoác" dùng <code>f_unaccent</code> + <code>pg_trgm</code> , có dùng index.	BE2	0,5 ngày
S7 — Lưu ảnh	Bản Local chạy thật (nén ảnh, xoá EXIF); bản S3 chạy qua cùng interface với MinIO. Thử với nơi lưu thật khi chốt C12 , sau khi web ổn định.	BE2	1 ngày
S8 — State machine	Trạng thái Request bằng enum + bảng chuyển hợp lệ; chuyển sai thì ném lỗi 409; có test.	BE3	1 ngày
S9 — Chat	Chọn polling hay WebSocket (STOMP); làm bản đơn giản nhất chạy được với FE.	BE3	1 ngày
S10 — Job nền	<code>@Scheduled</code> chạy lặp 2 lần không gây tác dụng kép.	BE3	0,5 ngày
S11 — Hosting PostgreSQL	Tạo thử database trên 1–2 dịch vụ managed theo nguyên tắc C06, kiểm tra bắt được <code>unaccent</code> , <code>pg_trgm</code> ; ghi giới hạn gói miễn phí.	BE4	0,5 ngày

Gợi ý cho S9 — chat

Với 6 tuần còn lại, bản đầu nên dùng **polling** (FE gọi API lấy tin mới mỗi 3–5 giây): đơn giản, dễ test, đủ cho demo. WebSocket là nâng cấp khi còn thời gian. Bộ lọc kiểm duyệt nằm ở service nên đổi cách truyền tin sau này không ảnh hưởng.

3.8. Bước 9 — Backlog theo tuần

- Mỗi FR mức Must của sprint tuần đó thành một issue trên GitHub Projects, ghi mã FR, BR liên quan và tiêu chí nghiệm thu (lấy từ luồng thay thế của use case).
- Issue đủ nhỏ để làm trong 1–2 ngày; lớn hơn thì tách. Issue chạm hai module thì tách thành hai issue, nối bằng service cửa ngõ (mục 4.7).

- Gắn nhãn module (trùng tên package) và người phụ trách; cột Todo / In progress / Review / Done.
- Mỗi tuần là một sprint (thứ Hai lập kế hoạch, Chủ nhật demo nội bộ). Sprint tuần 4: đăng ký + OTP + đăng nhập; danh mục, khu vực; tải ảnh và tạo bài (chưa kiểm duyệt); gửi yêu cầu; cấu hình trong `website_attributes`. Các sprint sau theo SRS chương 16.

4. Cấu trúc thư mục và cách 4 người code song song

Mục tiêu của chương này: **mỗi người làm trong thư mục của mình, gọi sang phần của người khác qua một số ít hàm đã thống nhất trước**, và những file dùng chung được giữ ở mức tối thiểu. Làm đúng thì phần lớn PR chỉ chạm một thư mục, merge hằng ngày gần như không có xung đột.

4.1. Năm nguyên tắc

1. **Chia theo module nghiệp vụ, không chia theo tầng.** Không có thư mục `controllers/` chung cho cả dự án; mỗi module có controller, service, repository, entity, dto của riêng nó.
2. **Một module — một người sở hữu** (bảng mục 4.6). Người khác sửa được, nhưng PR phải qua người sở hữu duyệt.
3. **Gọi chéo module chỉ qua service công khai hoặc sự kiện.** Không inject repository của module khác, không đọc entity của module khác để sửa.
4. **Giữa các module chỉ lưu id.** Item lưu `donorId` kiểu `Long`, không `@ManyToOne User`. Quan hệ JPA chỉ dùng bên trong một module (vd `Item` ↔ `ItemMediaMapping`).
5. **Dùng chung càng ít càng tốt.** `common/` chỉ chứa thứ không thuộc nghiệp vụ nào; mọi thứ khác nằm trong module. Khi phân vân, đặt vào module.

4.2. Cấu trúc repo

```
swp391_shareloop_team1_fa26/
├── .github/
│   ├── workflows/
│   │   ├── backend.yml          # build + spotless:check + test khi backend/** đổi
│   │   └── frontend.yml
│   ├── CODEOWNERS              # ai duyệt thư mục nào (mục 4.11)
│   └── pull_request_template.md
├── backend/                    # Spring Boot – 4 BE (mục 4.3)
├── frontend/                   # React + Vite – FE
├── docs/
│   ├── srs/                   # SRS, Luồng hoạt động & ERD, Lộ trình (docx/pdf)
│   ├── api/                   # contract theo module: auth.md, item.md ...
│   └── decisions/              # biên bản chốt: mã C, nội dung, ngày
├── AGENTS.md                  # hướng dẫn chung cho mọi trợ lý AI (mục 5.4)
├── CLAUDE.md                  # một dòng: @AGENTS.md
├── .gemini/settings.json       # cho Gemini CLI đọc AGENTS.md
├── .editorconfig               # thực lệ, mã hoá, xuống dòng cho mọi IDE
├── .gitattributes              # xuống dòng LF (quan trọng với Windows)
├── .gitignore
└── README.md                  # cách chạy + mục "Quy ước"
```

4.3. Cấu trúc thư mục backend

```
backend/
├── pom.xml                    # BE1 giữ; thêm dependency bằng PR riêng
├── mvnw, mvnw.cmd, .mvn/      # Maven Wrapper: mọi máy build cùng một bản Maven
├── docker-compose.yml         # PostgreSQL 16, Mailpit, MinIO
├── .env.example               # mẫu biến môi trường; .env thật không commit
├── http/                      # file .http gọi thư API, mỗi module một file
│   ├── auth.http             item.http   wallet.http   request.http   admin.http
├── src/
│   └── main/
```

```

├── java/com/shareloop/
│   ├── ShareLoopApplication.java
│   ├── common/           # nền dùng chung, không có nghiệp vụ      (BE1)
│   ├── config/           # cấu hình Spring                        (BE1)
│   ├── integration/      # gọi hệ thống bên ngoài          (theo thư mục con)
│   ├── auth/             # đăng ký, OTP email, đăng nhập, JWT    (BE1)
│   ├── user/             # hồ sơ, đổi số điện thoại            (BE1)
│   ├── wallet/           # ví Credit, nạp tiền, số Credit        (BE1)
│   ├── listingfee/       # phí đăng bài, phí sửa, gia hạn, đầy bài (BE1)
│   ├── catalog/          # danh mục, thuộc tính động, khu vực    (BE2)
│   ├── media/            # tải ảnh, media_files                (BE2)
│   ├── item/             # bài đăng                            (BE2)
│   ├── moderation/       # lexicon, regex, AI sàng lọc / chat    (BE2)
│   ├── search/           # tìm kiếm, AI trợ lý tìm đồ           (BE2)
│   ├── request/          # Request, state machine, lịch, giao nhận (BE3)
│   ├── chat/            # tin nhắn, thẻ bài, đổi món đề nghị     (BE3)
│   ├── notification/     # thông báo trong app + email giao dịch  (BE3)
│   ├── setting/          # cấu hình, lexicon, checklist          (BE4)
│   ├── review/           # hàng chờ duyệt, duyệt / từ chối      (BE4)
│   ├── report/           # báo cáo, khiếu nại, tranh chấp        (BE4)
│   ├── reputation/       # TrustStars, điểm, hạng               (BE4)
│   ├── admin/            # khoá tài khoản, báo cáo tài chính     (BE4)
│   ├── audit/            # activity_logs                       (BE4)
│   └── resources/        # mục 4.10
└── test/                # cùng cây package với main (mục 4.10)

```

Ba thư mục nền được chia nhỏ như sau:

```

common/
├── api/           PageResponse, ErrorResponse
├── exception/     ErrorCode (interface), CommonErrorCode, BusinessException,
                  GlobalExceptionHandler
├── entity/        BaseEntity (id + 6 cột chuẩn), SliceAudit
└── security/      CurrentUserId (annotation), CurrentUserArgumentResolver

config/
├── SecurityConfig.java    # quy tắc URL theo quy ước, không sửa khi thêm API
├── JwtConfig.java         # JwtEncoder / JwtDecoder
├── OpenApiConfig.java     # Swagger
├── AsyncConfig.java       # @Async cho AI và email sau commit
├── SchedulingConfig.java  # bật @Scheduled
├── JpaAuditingConfig.java
└── ClockConfig.java       # bean Clock, test chỉnh được giờ

integration/
├── payment/ (BE1) PaymentGatewayClient, VnPayClient, MockPaymentClient
├── mail/    (BE1) EmailSender, SmtplibEmailSender
├── ai/      (BE2) AiClient, <Provider>AiClient, AiProperties
└── storage/ (BE2) StorageService, LocalStorageService, S3StorageService

```

4.4. Bên trong một module

Mọi module dùng cùng một khuôn. Thư mục nào module không cần thì không tạo. Ví dụ module `item`:

```

item/
├── controller/
│   ├── ItemController.java          # /api/v1/items/**
│   └── MyItemController.java        # /api/v1/me/items
├── service/
│   ├── ItemService.java             # ghi: tạo, sửa, đổi status (cửa ngõ)
│   └── ItemQueryService.java         # đọc: chi tiết, danh sách theo bộ lọc
├── repository/
│   ├── ItemRepository.java
│   ├── ItemMediaMappingRepository.java
│   └── ItemAttributeMappingRepository.java
├── entity/
│   ├── Item.java                   # entity chính của bảng items
│   ├── ItemMediaMapping.java
│   ├── ItemAttributeMapping.java
│   ├── ItemStatus.java             # enum + canTransition(from, to)
│   ├── OfferType.java
│   └── ItemCondition.java
├── dto/
│   ├── CreateItemRequest.java       # record + @Valid
│   ├── UpdateItemRequest.java
│   ├── ItemResponse.java
│   └── ItemSummaryResponse.java
├── mapper/
│   └── ItemMapper.java              # entity ↔ DTO, viết tay
├── event/
│   └── ItemSubmittedEvent.java       # moderation nghe sự kiện này để gọi AI
├── job/
│   └── ItemExpiryJob.java            # APPROVED quá expire_at → EXPIRED
└── ItemErrorCode.java               # ITEM_NOT_FOUND, ITEM_NOT_EDITABLE, ...

```

Loại	Quy ước đặt tên	Ví dụ
Controller	<Tên>Controller; API của Admin tách riêng Admin<Tên>Controller nhưng vẫn nằm trong module sở hữu dữ liệu	AdminCategoryController trong catalog/, path /api/v1/admin/categories
Service	<Tên>Service; hàm public là API cho module khác, hàm nội bộ để package-private	PostFeeService.chargeHeldFee(itemId)
DTO	record, hậu tố Request / Response	CreateItemRequest, WalletResponse
Enum trạng thái	<Tên>Status kèm canTransition(from, to)	RequestStatus, ItemStatus
Sự kiện	Thì quá khứ, hậu tố Event; người nghe hậu tố Listener	ItemSubmittedEvent, ItemScreeningListener
Job	Hậu tố Job, nằm trong module sở hữu dữ liệu	PaymentOrderExpiryJob trong wallet/job/
Mã lỗi	Enum <Module>ErrorCode implements ErrorCode, tiền tố tên module	WALLET_INSUFFICIENT_BALANCE
Test	Unit ...Test; test controller ...ControllerTest; test có DB thật ...IT	CreditServiceIT

Loại	Quy ước đặt tên	Ví dụ
Migration	V<yyyyMMddHHmm>__<module>_<mô_tả>.sql (mục 4.9)	V202610061530__item_add_view_index.sql

4.5. Module nào cần khi nào

Không phải module nào cũng bắt đầu ngay. Thứ tự dưới đây bám lịch SRS chương 16, để mỗi tuần mọi người đều có việc trong thư mục của mình:

Tuần	BE1	BE2	BE3	BE4
4	common, config, auth, user	catalog, media, item (tạo bài)	request (enum, gửi yêu cầu)	setting, khung admin, audit
5	wallet, integration/payment, listingfee (giữ / trừ / nhà)	moderation tầng 1	request (chọn người), chat (polling)	review
6	listingfee (sửa, gia hạn)	search, moderation tầng 2 (AI sàng lọc)	request (lich, liên lạc, giao nhận), notification, job	report, reputation
7	listingfee (đẩy bài), deploy	search (AI trợ lý), AI chat tầng 2	chat (thẻ bài, đổi món)	admin (báo cáo tài chính, khoá tài khoản)

4.6. Module — người sở hữu — dữ liệu được ghi

Bảng này khớp với mục 2 của tài liệu Luồng hoạt động & ERD. Cột "Dữ liệu được ghi" là quy tắc **một nhóm cột — một module ghi**: module khác muốn đổi các cột đó phải gọi service của module sở hữu.

Package	Người i	Service chính	Dữ liệu được ghi
auth, user	BE1	AuthService, OtpService, JwtService, UserService	users: mật khẩu, trạng thái, otp_*, hồ sơ, phone, pending_phone
wallet	BE1	CreditService, PaymentService	payment_orders, credit_ledger; users: credit_balance, held_credit, has_topped_up
listingfee	BE1	PostFeeService, RenewService, BoostService	items: post_fee_paid, pending_fee*, fee_state, free_edit_used, approved_edit_count, expire_at, boosted_until
catalog, media	BE2	CategoryService, AreaService, MediaService	item_categories, item_attributes, areas, media_files
item	BE2	ItemService, ItemQueryService	items: nội dung, status; item_media_mappings, item_attribute_mappings

Package	Người i	Service chính	Dữ liệu được ghi
moderation, search	BE2	ContentModerationService, ItemScreeningService, SearchService, AiAssistantService	items: ai_risk, ai_result; users: free_ai_used, free_ai_date
request, chat, notification	BE3	RequestService, LogisticsService, HandoverService, ChatService, NotificationService	requests, request_media_mappings, messages, notifications
setting, review	BE4	ConfigService, LexiconService, ChecklistService, ReviewService	website_attributes; items: review_snapshot, reject_reason, approved_at, auth_status
report, reputation, admin, audit	BE4	ReportService, ReputationService, AdminUserService, FinanceReportService, ActivityLogService	reports, activity_logs; users: trust_stars, total_points, rank_level, points_*, clean_streak, grace_star_used

4.7. Gọi chéo module: service cửa ngõ

Được làm	Không được làm
Inject và gọi hàm <code>public</code> của <code><Tên>Service</code> module khác	Inject repository của module khác
Nghe sự kiện của module khác (<code>@TransactionalEventListener</code>) để làm việc sau commit	Sửa trực tiếp entity lấy từ module khác
Dùng enum và DTO công khai của module khác (<code>ItemStatus</code> , <code>ModerationResult</code>)	<code>@ManyToOne</code> / <code>@OneToMany</code> sang entity của module khác
Lưu id của bản ghi module khác (<code>Long donorId</code>)	Đặt class nghiệp vụ vào <code>common/</code> cho "tiện dùng chung"

Những hàm dưới đây là **điểm nối giữa người này với người kia**. Người sở hữu viết chữ ký hàm và thân rỗng (ném `UnsupportedOperationException` hoặc trả giá trị giả), merge ngay trong tuần 4. Người gọi code tiếp được mà không phải chờ; người sở hữu làm phần thân sau, không ai sửa file của ai.

Service cửa ngõ	Chủ	Hàm chính	Ai gọi
wallet.CreditService	BE1	hold, release, chargeHeld, spend(userId, amount, type, itemId), topUp	listingfee, search
listingfee.PostFeeService	BE1	quoteEdit, holdPostFee, holdEditFee, chargeHeldFee, releaseHeldFee	item, review
item.ItemService	BE2	markApproved, markRejected, reserve, release, markTraded, hide, unhide, remove, hideAllOfUser	review, request, reputation, report, admin

Service cửa ngõ	Chủ	Hàm chính	Ai gọi
<code>media.MediaService</code>	BE2	<code>attach(mediaIds, userId)</code> (UPLOADED → ATTACHED, kiểm người tải), <code>deactivate</code> , <code>remove</code>	item, chat, request, user, admin
<code>user.UserService</code>	BE1	<code>lock</code> , <code>unlock</code> (đổi <code>is_active</code>), <code>findContact</code>	admin, reputation, request
<code>moderation.ContentModerationService</code>	BE2	<code>checkPost(...)</code> , <code>checkMessage(...)</code> → <code>ModerationResult</code>	item, chat, request
<code>setting.ConfigService</code>	BE4	<code>getInt(ConfigKey)</code> , <code>getDecimal(...)</code> , <code>lexicon()</code> , <code>checklist()</code>	mọi module
<code>audit.ActivityLogService</code>	BE4	<code>log(type, actorId, targetType, targetId, data, reason)</code>	mọi module
<code>notification.NotificationService</code>	BE3	<code>notify(userId, type, title, body, link)</code> , <code>email(...)</code>	mọi module
<code>reputation.ReputationService</code>	BE4	<code>onCompleted(requestId)</code> , <code>deductStar(...)</code> , <code>grantGraceStar(...)</code>	request, report

Ví dụ một thao tác đi qua ba module mà mỗi người chỉ sửa file của mình:

```
// review/service/ReviewService.java – BE4
@Service
@RequiredArgsConstructor
public class ReviewService {
    private final ItemService itemService;           // item (BE2)
    private final PostFeeService postFeeService;     // listingfee (BE1)
    private final ItemReviewRepository reviews;      // lát cắt của chính review
    private final ActivityLogService activityLog;     // audit (BE4)

    @Transactional
    public void approve(long itemId, long adminId, ReviewDecisionRequest req) {
        postFeeService.chargeHeldFee(itemId);        // trừ phí đang giữ
        itemService.markApproved(itemId);            // đổi status qua chủ sở hữu
        reviews.lockById(itemId).orElseThrow()
            .recordDecision(req.answers(), null);    // review_snapshot
        activityLog.log(REVIEW, adminId, ITEM, itemId, req.answers(), req.note());
    }
}
```

4.8. Bảng users và items: entity "lát cắt"

Hai bảng rộng `users` và `items` được nhiều module ghi (mỗi module một nhóm cột). Nếu cả nhóm dùng chung một file `User.java` thì ai cũng sửa file đó và xung đột liên tục. Cách làm: **module sở hữu nhóm cột khai báo một entity nhỏ chỉ map id và các cột của mình**, trở vào cùng bảng.

```
// wallet/entity/WalletAccount.java – BE1: chỉ các cột tiền của bảng users
@Entity
@Table(name = "users")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@SQLRestriction("is_deleted = false")
```

```

public class WalletAccount extends SliceAudit { // map updated_at, updated_by
    @Id private Long id;
    @Column(name = "credit_balance") private long creditBalance;
    @Column(name = "held_credit") private long heldCredit;
    @Column(name = "has_topped_up") private boolean hasToppedUp;

    public long available() { return creditBalance - heldCredit; }

    void hold(long amount) { // package-private: chỉ CreditService gọi
        if (available() < amount)
            throw new BusinessException(WalletErrorCode.WALLET_INSUFFICIENT_BALANCE);
        heldCredit += amount;
    }
}

// wallet/repository/WalletAccountRepository.java
public interface WalletAccountRepository extends JpaRepository<WalletAccount, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE) // SELECT ... FOR UPDATE
    @Query("select w from WalletAccount w where w.id = :id")
    Optional<WalletAccount> lockById(@Param("id") Long id);
}

```

Entity	Package	Bảng	Cột map (ngoài id)
User (entity chính, tạo bản ghi)	user	users	email, mật khẩu, hồ sơ, phone, status, otp_*, kế thừa BaseEntity (6 cột chuẩn)
WalletAccount	wallet	users	credit_balance, held_credit, has_topped_up
AiQuota	search	users	free_ai_used, free_ai_date
ReputationProfile	reputation	users	trust_stars, total_points, rank_level, points_today, points_date, clean_streak, grace_star_used
Item (entity chính, tạo bản ghi)	item	items	nội dung, status, donor_id, category_id, area_id, view_count, thời điểm
ItemFee	listingfee	items	post_fee_paid, pending_fee_type, pending_fee, fee_state, free_edit_used, approved_edit_count, expire_at, boosted_until
ItemAiFlag	moderation	items	ai_risk, ai_result
ItemReview	review	items	review_snapshot, reject_reason, approved_at, auth_status

Ba luật khi dùng entity lát cắt

- Chỉ entity chính (User, Item) được tạo bản ghi; lát cắt chỉ đọc và sửa.
- Không map một cột ở hai entity (trừ id, và updated_at, updated_by mà mọi lát cắt cùng map qua SliceAudit để ghi đúng người sửa). Hibernate kiểm tra từng entity với schema khi khởi động (ddl-auto: validate), nên map sai tên cột sẽ báo lỗi ngay.
- Cần khoá dòng thì khoá qua lát cắt của mình (lockById), trong transaction của service sở hữu. Tiền luôn khoá qua WalletAccount.

4.9. Migration Flyway khi 4 người cùng sửa schema

- Bốn file khởi tạo V1–V4 như mục 3.3. Sau đó mọi file mới đặt version theo **thời điểm tạo**: `V<yyyyMMddHHmm>__<module>_<mô_tả>.sql`, ví dụ `V202610061530__item_add_view_index.sql`. Hai người gần như không bao giờ trùng số, và số này luôn lớn hơn 1–4.
- Ở profile `dev` bật `spring.flyway.out-of-order: true`: nếu file của bạn có số nhỏ hơn file người khác đã chạy trên máy bạn (do merge sau), Flyway vẫn chạy nó. Profile `prod` để mặc định `false`.
- Không sửa file migration đã merge vào `develop`**. Viết sai thì tạo file mới để sửa. Nếu máy mình lỡ chạy bản nháp, xóa database dev (`docker compose down -v`) rồi chạy lại.
- Dữ liệu mẫu cho dev (tài khoản thử, bài mẫu) để trong `db/dev-data/` dạng file lập lại được `R__dev_seed.sql`; chỉ nạp ở profile `dev`, không lên production.
- Mọi file trong `db/migration/` cần BE1 duyệt (CODEOWNERS), vì schema đã chốt: file nào làm lệch ERD thì phải ghi thêm vào mục 7.8 tài liệu ERD.

4.10. Thư mục resources và test

```
src/main/resources/
├─ application.yml           # chung; mỗi module một khối app.<module>
├─ application-dev.yml
├─ application-prod.yml      # chỉ ${BIEN_MOI_TRUONG}, không chứa bí mật
├─ db/
│   └─ migration/           # V1__init.sql ... V4__..., V<thời gian>__...
│       └─ dev-data/        # R__dev_seed.sql – chỉ profile dev
├─ templates/mail/          # otp.html, contact-revealed.html, ... (BE1, BE3)
└─ prompts/                 # prompt LLM (BE2), không viết trong mã:
                             # item-screening.txt, chat-moderation.txt ...

src/test/
├─ java/com/shareloop/
│   └─ support/
│       ├── IntegrationTest.java # @SpringBootTest + container PostgreSQL
│       └─ TestDataFactory.java  # tạo nhanh user, item, request cho test
│   └─ architecture/
│       └─ ModuleBoundaryTest.java # (tuỳ chọn) ArchUnit kiểm ranh giới
│   └─ wallet/CreditServiceIT.java
│   └─ request/RequestStatusTest.java
│   └─ ...                    # cùng package với mã được test
└─ resources/application-test.yml
```

`application.yml` là file dùng chung duy nhất mà ai cũng cần thêm cấu hình. Quy ước: mỗi module một khối riêng (`app.wallet.*`, `app.ai.*`, `app.storage.*`), người sở hữu module chỉ sửa trong khối của mình, thêm dòng ở cuối khối; như vậy Git tự gộp được hầu hết trường hợp.

4.11. CODEOWNERS và những file "nóng"

Mỗi vùng ghi **hai người**: người đầu là chủ, người sau là dự phòng (BE1 ↔ BE3, BE2 ↔ BE4). Khi bật "bắt buộc review của code owner", tác giả PR không tự duyệt được PR của mình, nên vùng chỉ có một người sẽ kệt PR của chính người đó.

```
# .github/CODEOWNERS
# CODEOWNERS - nguoi duoc hoi review tu dong theo duong dan.
# Dong SAU thang dong TRUOC. Moi vung ghi 2 nguoi vi tac gia PR khong tu duyệt
```

```
# duoc PR cua minh: nguoi dau la chu, nguoi sau la du phong.
# Cap du phong: BE1 <-> BE3, BE2 <-> BE4.
*
/frontend/ @ariso-29 @dwargon73-sketch @My-Mieu @ariso-29
# backend chung (docker-compose, http/, test/support...): ca 4 BE
/backend/ @ariso-29 @Nguyentri2531 @dwargon73-sketch @kopslnbtram2110

# ----- BE1: nen va file dung chung -----
/backend/pom.xml @ariso-29 @dwargon73-sketch
/backend/**/shareloop/common/ @ariso-29 @dwargon73-sketch
/backend/**/shareloop/config/ @ariso-29 @dwargon73-sketch
/backend/src/main/resources/db/ @ariso-29 @dwargon73-sketch
# ----- BE1: nghiep vu -----
/backend/**/shareloop/auth/ @ariso-29 @dwargon73-sketch
/backend/**/shareloop/user/ @ariso-29 @dwargon73-sketch
/backend/**/shareloop/wallet/ @ariso-29 @dwargon73-sketch
/backend/**/shareloop/listingfee/ @ariso-29 @dwargon73-sketch
/backend/**/integration/payment/ @ariso-29 @dwargon73-sketch
/backend/**/integration/mail/ @ariso-29 @dwargon73-sketch
# ----- BE2 -----
/backend/**/shareloop/catalog/ @Nguyentri2531 @kopslnbtram2110
/backend/**/shareloop/media/ @Nguyentri2531 @kopslnbtram2110
/backend/**/shareloop/item/ @Nguyentri2531 @kopslnbtram2110
/backend/**/shareloop/moderation/ @Nguyentri2531 @kopslnbtram2110
/backend/**/shareloop/search/ @Nguyentri2531 @kopslnbtram2110
/backend/**/integration/ai/ @Nguyentri2531 @kopslnbtram2110
/backend/**/integration/storage/ @Nguyentri2531 @kopslnbtram2110
/backend/src/main/resources/prompts/ @Nguyentri2531 @kopslnbtram2110
# ----- BE3 -----
/backend/**/shareloop/request/ @dwargon73-sketch @ariso-29
/backend/**/shareloop/chat/ @dwargon73-sketch @ariso-29
/backend/**/shareloop/notification/ @dwargon73-sketch @ariso-29
# ----- BE4 -----
/backend/**/shareloop/setting/ @kopslnbtram2110 @Nguyentri2531
/backend/**/shareloop/review/ @kopslnbtram2110 @Nguyentri2531
/backend/**/shareloop/report/ @kopslnbtram2110 @Nguyentri2531
/backend/**/shareloop/reputation/ @kopslnbtram2110 @Nguyentri2531
/backend/**/shareloop/admin/ @kopslnbtram2110 @Nguyentri2531
/backend/**/shareloop/audit/ @kopslnbtram2110 @Nguyentri2531
```

File dùng chung	Vì sao dễ xung đột	Cách làm
<code>pom.xml</code>	Ai cũng muốn thêm thư viện	BE1 giữ. Thêm dependency bằng một PR riêng nhỏ, merge trong ngày; phiên bản khai báo trong <code><properties></code>
<code>application.yml</code>	Cấu hình của mọi module	Mỗi module một khối <code>app.<module>.*</code> ; bí mật để <code>\${BIEN}</code>
<code>SecurityConfig</code>	Phân quyền URL	Quy ước URL cố định (mục 3.5.4) nên không cần sửa; quyền chi tiết dùng <code>@PreAuthorize</code> trong controller của module
Mã lỗi	Một enum chung ai cũng thêm dòng	Mỗi module một enum <code><Module>ErrorCode</code>
<code>ConfigKey</code>	Khoá cấu hình của mọi module	BE4 tạo một lần đủ các khoá ở mục 7.10 ERD; thêm khoá mới = một PR nhỏ kèm migration
Migration	Trùng số version	Version theo thời gian (mục 4.9)

File dùng chung	Vì sao dễ xung đột	Cách làm
<code>common/</code> , <code>config/</code>	Lớp nền	BE1 viết xong trong tuần 4, sau đó hầu như không sửa; cần thêm thì mở issue trước
<code>docs/api/</code>	Contract	Mỗi module một file

Một ngày làm việc điển hình của một bạn BE

- Sáng: `git switch develop && git pull`, rồi `git switch -c feature/<module>-<việc>`.
- Làm trong thư mục module của mình; cần gì của người khác thì gọi service cửa ngõ, thiếu hàm thì nhắn người sở hữu thêm chữ ký hàm.
- Trước khi đẩy: `./mvnw spotless:apply` rồi `./mvnw verify`.
- Mở PR nhỏ vào `develop`; code owner được hỏi review tự động. Merge trong ngày, không để PR sống qua 2 ngày.

5. Công cụ phát triển: IDE và trợ lý AI

Nhóm được tự chọn IDE (IntelliJ IDEA, Antigravity, VS Code) và trợ lý AI (Claude Code, Codex, Gemini...). Tự do chọn công cụ chỉ an toàn khi **những thứ quyết định kết quả nằm trong repo chứ không nằm trong IDE**: phiên bản Java và Maven, định dạng mã, cách xuống dòng, và hướng dẫn cho trợ lý AI.

5.1. Cài đặt chung (cả 4 BE)

Công cụ	Phiên bản / cách cài	Ghi chú
JDK	Temurin 21 (LTS)	Cả nhóm cùng một bản. IntelliJ tải được JDK ngay trong IDE; VS Code và Antigravity đọc <code>JAVA_HOME</code> hoặc cấu hình <code>java.jdt.ls.java.home</code>
Maven	Không cài riêng; dùng <code>./mvnw</code> (Windows: <code>mvnw.cmd</code>)	Maven Wrapper khoá phiên bản Maven trong repo
Docker Desktop	Bản mới nhất	Chạy PostgreSQL, Mailpit, MinIO bằng <code>docker compose up -d</code> ; Testcontainers cũng cần Docker
Git	Bản mới nhất	Không cần chỉnh <code>core.autocrlf</code> khi repo đã có <code>.gitattributes</code>
Gọi thử API	File <code>.http</code> trong <code>backend/http/</code>	Chạy được trong IntelliJ (HTTP Client) và VS Code / Antigravity (extension REST Client); hoặc dùng Postman
Xem database	DBeaver, pgAdmin hoặc công cụ Database của IntelliJ	Kết nối <code>localhost:5432</code> , user/pass <code>shareloop</code>

5.2. Thiết lập từng IDE

IDE	Cài thêm	Lưu ý
IntelliJ IDEA	Plugin Lombok (thường có sẵn); bật *Annotation Processing*; plugin Spotless hoặc chạy lệnh Maven	Mở thư mục <code>backend/</code> như project Maven. Tắt "Optimize imports on the fly" nếu nó làm đổi thứ tự import khác Spotless
VS Code	*Extension Pack for Java*, *Spring Boot Extension Pack*, *Lombok Annotations Support* (nếu bản Java extension chưa gộp sẵn), *REST Client*, *Docker*	Formatter mặc định của VS Code khác IntelliJ; không dựa vào "Format Document", dùng Spotless
Antigravity	Các extension Java như VS Code (cài từ kho extension của Antigravity); nếu thiếu extension nào thì chạy / debug bằng VS Code hoặc IntelliJ	Nên là VS Code nên thiết lập giống VS Code. Agent của Antigravity đọc <code>AGENTS.md</code> ở gốc repo (mục 5.4)

Không commit thư mục cấu hình riêng của IDE (`.idea/`, `*.iml`, `.vscode/` — trừ `.vscode/extensions.json` để gợi ý extension cho người mới). Thêm các mục này vào `.gitignore` ngay trong skeleton.

5.3. Giữ mã đồng nhất giữa các IDE

Ba IDE định dạng mã khác nhau (thụt lề, xuống dòng, thứ tự import). Nếu không thống nhất, mỗi lần một người mở file của người khác và lưu lại là cả file bị đổi, PR phình to và xung đột giả. Ba lớp bảo vệ:

- 1. `.editorconfig` ở gốc repo: UTF-8, xuống dòng LF, Java thụt 4 dấu cách, YAML/JSON/TS thụt 2. Cả ba IDE đều đọc file này.
- 2. `.gitattributes`: `* text=auto eol=lf`, riêng `*.cmd` và `*.bat` giữ CRLF, ảnh và `.jar` đánh dấu `binary`. Quan trọng với máy Windows: không có file này, Git có thể đổi toàn bộ file sang CRLF và `mvnw` không chạy được trên CI.
- 3. **Spotless** trong `pom.xml` là "trọng tài" duy nhất: `./mvnw spotless:apply` định dạng lại mã, CI chạy `spotless:check`. IDE nào cũng được, miễn là trước khi commit đã chạy lệnh này.

```
<!-- pom.xml, trong <build><plugins> -->
<plugin>
  <groupId>com.diffplug.spotless</groupId>
  <artifactId>spotless-maven-plugin</artifactId>
  <version>${spotless.version}</version>    <!-- khai báo trong <properties> -->
  <configuration>
    <java>
      <palantirJavaFormat/>                <!-- 4 dấu cách, dòng 120 ký tự -->
      <removeUnusedImports/>
      <importOrder/>
    </java>
  </configuration>
</plugin>

# .gitattributes
* text=auto eol=lf
*.cmd text eol=crlf
*.bat text eol=crlf
*.png binary
*.jpg binary
*.jar binary
```

5.4. Trợ lý AI: một bộ hướng dẫn cho mọi công cụ

Mỗi trợ lý AI đọc một file hướng dẫn khác nhau. Nhóm viết **một file `AGENTS.md` duy nhất** ở gốc repo và trở các công cụ còn lại về nó, để Claude Code, Codex, Gemini và Antigravity cùng tuân theo một cấu trúc thư mục và một bộ luật.

Công cụ	File hướng dẫn nó đọc	Cách trở về AGENTS.md
Codex (CLI, extension IDE)	AGENTS.md	Đọc trực tiếp, không cần làm gì
Antigravity	AGENTS.md, GEMINI.md, .agents/rules/*.md	Đọc trực tiếp AGENTS.md
Claude Code	CLAUDE.md	Tạo CLAUDE.md chỉ có một dòng @AGENTS.md (cú pháp import của Claude Code)
Gemini CLI / Gemini Code Assist	GEMINI.md (mặc định)	Tạo .gemini/settings.json: { "context": { "fileName": ["AGENTS.md", "GEMINI.md"] } }

Nội dung gợi ý cho **AGENTS.md** (ngắn, chỉ ghi điều AI hay làm sai):

```
# ShareLoop – hướng dẫn cho trợ lý AI

Trả lời và giải thích bằng tiếng Việt. Tên class, biến, commit message viết tiếng Anh.

## Git, commit và ghi công (luật cứng)
- KHÔNG chạy git commit, git push, git merge, git rebase, gh pr create/merge.
  Người làm tự xem diff, tự chạy test và tự commit dưới tên mình.
- KHÔNG ghi công AI ở bất kỳ đâu: không Co-Authored-By, không "Generated with/by",
  không tên công cụ AI trong commit message, mô tả PR, comment, @author,
  README hay tài liệu.
- Khi được nhờ soạn commit message hoặc mô tả PR: chỉ viết nội dung
  dạng type(scope): mô tả, không thêm dòng ký tên nào.

## Dự án
- Monorepo: backend/ (Java 21, Spring Boot 4.1.1, Maven, PostgreSQL 16, Flyway),
  frontend/ (React + Vite; có AGENTS.md riêng, làm trong frontend/ thì theo file đó).
- Spring Boot khoá ở 4.1.1, không tự đổi phiên bản. Viết theo Spring Boot 4:
  spring-boot-starter-webmvc, spring-boot-starter-flyway, Jackson 3 (tools.jackson),
  @MockitoBean. Không dùng starter "classic".
- Đặc tả: docs/srs/ (SRS v1.0; Luồng hoạt động & ERD – 17 bảng ĐÃ CHÔT;
  Lộ trình triển khai Backend). Không tự thêm bảng.
- Contract API: docs/api/<module>.md. Đọc file của module trước khi viết controller.

## Lệnh (chạy trong backend/)
- Database dev: docker compose up -d
- Định dạng: ./mvnw spotless:apply          Build + test: ./mvnw verify

## Cấu trúc (Lộ trình, chương 4)
- Package theo module: com.shareloop.<module>/
  {controller, service, repository, entity, dto, mapper, event, job}
- Chỉ sửa trong module của task. Gọi module khác qua <Tên>Service;
  KHÔNG inject repository của module khác.
- Giữa các module chỉ lưu id (Long), không @ManyToOne sang entity module khác.
- Bảng users/items: mỗi module map nhóm cột của mình bằng entity lát cắt;
  không map một cột ở hai entity (trừ updated_at, updated_by qua SliceAudit).

## Cột chuẩn và xoá mềm (ERD mục 7.11)
- Entity bảng chính và bảng danh mục kế thừa common.entity.BaseEntity
  (created_at, updated_at, created_by, updated_by, is_active, is_deleted),
  gán @SQLRestriction("is_deleted = false"); status là enum riêng từng entity.
- Không xoá cứng: gọi markDeleted(). Ẩn / khoá: is_active = false, không đổi status.
- Không tự gán created_by, updated_by, created_at, updated_at: JPA Auditing lo.

## Luật bắt buộc
- Tiên chỉ đổi trong wallet.CreditService, có khoá dòng và ghi credit_ledger.
- Trạng thái Request/Item chỉ đổi qua service sở hữu, luôn kiểm canTransition().
- Mọi con số nghiệp vụ đọc ConfigService(ConfigKey), không viết cứng.
- Không trả entity ra API; DTO là record có @Valid.
- Lỗi nghiệp vụ: throw BusinessException(<Module>ErrorCode.X).
- Schema chỉ đổi bằng migration MỚI: V<yyyyMMddHHmm>__<module>_<mo_ta>.sql;
  không sửa migration cũ.
- Service có business rule phải có test; test DB dùng Testcontainers, không dùng H2.

## Không được làm
- Không đổi pom.xml, SecurityConfig, common/, config/ trừ khi task yêu cầu rõ.
```

- Không đọc, tạo hay commit .env, API key, mật khẩu.
- Không tắt hoặc xoá test để build xanh.

Cách dùng trợ lý AI trong nhóm

- **Một task — một nhánh — một phiên AI.** Mở phiên mới cho mỗi issue, nói rõ module và file contract liên quan.
- Để agent ở chế độ **hỏi trước khi chạy lệnh hoặc sửa file ngoài module**. Không cho agent tự push hay tự merge vào **develop**.
- Mã chạm tiền (**wallet**, **listingfee**) và chuyển trạng thái (**request**, **item**): tự viết test trước, cho AI viết phần thân sau, rồi đọc lại từng dòng.
- Không dán API key, mật khẩu, dữ liệu thật của người dùng vào prompt.
- Commit dưới tên người làm; người mở PR phải giải thích được mọi dòng khi review hỏi.
- Dùng AI để hỏi "vì sao" và để viết test, không chỉ để sinh mã.

6. Kiến thức cần học

6.1. Phần cả nhóm backend đều phải nắm

Chủ đề	Nội dung cụ thể cần học	Vì sao dự án cần
Java 21	<code>record</code> (DTO), <code>switch</code> dạng biểu thức và pattern matching (state machine), text block (prompt, SQL trong test), <code>Optional</code> , Stream, exception checked/unchecked, <code>equals/hashCode</code>	Nền của mọi dòng mã. Virtual thread không bắt buộc dùng.
Spring Boot 4 căn bản	IoC, Dependency Injection, Bean, <code>@Component/@Service/@Repository</code> , <code>application.yml</code> , profile; khác biệt so với Boot 3 khi đọc tài liệu cũ (hộp mục 3.5.1)	Hiểu vì sao mã chạy được, cấu hình dev/test/prod; cũng là nội dung bài thi thực hành (Spring Boot 4.1.1).
Spring Web (REST)	<code>@RestController</code> , <code>@RequestMapping</code> , <code>@PathVariable</code> , <code>@RequestBody</code> , <code>ResponseEntity</code> , DTO, <code>@Valid</code> , <code>@RestControllerAdvice</code>	Viết API đúng contract, trả lỗi thống nhất.
Spring Data JPA / Hibernate	Entity, quan hệ trong một module, lazy/eager, lỗi N+1, phân trang, <code>@Transactional</code> , <code>@Query</code> , <code>@Lock(PESSIMISTIC_WRITE)</code> , nhiều entity trên một bảng (mục 4.8), JPA Auditing, <code>@MappedSuperclass</code> , <code>@SQLRestriction</code> cho xoá mềm	Ví Credit và trạng thái giao dịch phải đúng tuyệt đối.
PostgreSQL	JOIN/GROUP BY, index, CHECK/UNIQUE, partial index, transaction, <code>SELECT ... FOR UPDATE</code> , JSONB, <code>EXPLAIN</code>	Viết migration, query báo cáo, kiểm tra hiệu năng.
Flyway	Đặt tên version theo thời gian, <code>out-of-order</code> ở dev, không sửa migration đã chạy	Bốn người cùng sửa schema mà không vỡ database của nhau.
Spring Security + JWT	Filter chain, <code>UserDetailsService</code> , BCrypt, OAuth2 Resource Server (<code>JwtEncoder/JwtDecoder</code>), <code>@PreAuthorize</code> , CORS	Đăng nhập, phân quyền Admin, bảo vệ API.
Kiểm thử	JUnit 5, Mockito, <code>@MockitoBean</code> , <code>@WebMvcTest</code> + <code>MockMvc</code> , <code>@DataJpaTest</code> , Testcontainers PostgreSQL	SRS yêu cầu test cho mọi nhánh tiền và trạng thái.
Git & GitHub	Nhánh, rebase/merge, giải xung đột, PR, CODEOWNERS	Làm việc 5 người trên một repo.
Docker	Image, container, volume, Docker Compose, đọc log	Môi trường dev; đóng gói để deploy.
Công cụ nhóm	Spotless, <code>.editorconfig</code> , file <code>.http</code> , <code>AGENTS.md</code>	Nhiều IDE và trợ lý AI vẫn ra mã đồng nhất.

6.2. Phần học thêm theo module

Người	Học thêm	Dùng cho
BE1	Tích hợp cổng thanh toán (redirect, IPN, chữ ký HMAC-SHA512, idempotency); Spring Mail; thiết kế sổ cái và cân đối sổ dư; xử lý đồng thời, khoá dòng	Nạp Credit, phí đăng bài, phí sửa, gia hạn, đẩy bài, OTP

Người	Học thêm	Dùng cho
BE2	Regex Java, chuẩn hoá Unicode (<code>java.text.Normalizer</code>), xử lý tiếng Việt; gọi HTTP bằng <code>RestClient</code> có timeout và retry; prompt ép JSON và kiểm schema; <code>@Async</code> ; <code>pg_trgm</code> ; upload multipart, AWS SDK v2 S3	Kiểm duyệt nội dung, AI, tìm kiếm, ảnh
BE3	State machine bằng enum; <code>@Scheduled</code> và job idempotent; polling / WebSocket (STOMP) cho chat; sự kiện trong ứng dụng (<code>ApplicationEventPublisher</code> , <code>@TransactionalEventListener</code>)	Request, chat, thông báo, job nền
BE4	Truy vấn báo cáo (GROUP BY theo thời gian, <code>date_trunc</code>), xuất CSV; cấu hình key-value và cache cấu hình; phân quyền Admin; nhật ký kiểm toán	Trang Admin, báo cáo, cấu hình, khiếu nại, uy tín
FE	Axios interceptor gắn JWT và xử lý 401; đọc <code>code</code> lỗi để hiển thị; mock theo contract (MSW hoặc json-server); upload nhiều ảnh; polling	Nổi API

6.3. Cách học hiệu quả trong thời gian ngắn

- **Học bằng spike.** Không đọc hết khoá học rồi mới làm; học đúng phần cần cho spike của mình, làm spike, rồi quay lại học phần còn thiếu.
- **Nguồn nên dùng:** Spring Guides trên spring.io; tài liệu Spring Boot Reference (đúng phiên bản 4.1.x); Baeldung cho từng chủ đề; tài liệu PostgreSQL phần Tutorial và Indexes; tài liệu Testcontainers; tài liệu tích hợp VNPay sandbox.
- **Chia sẻ lại cho nhóm:** mỗi người trình bày 15 phút về spike của mình trong buổi họp tuần.
- **Dùng trợ lý AI để hỏi "vì sao"**, không chỉ để sinh mã. Mã chạy tiền và trạng thái phải tự hiểu được.

7. Những gì còn phải tìm hiểu

Câu hỏi	Cách tìm hiểu	Ảnh hưởng tới	Ai
Dùng nhà cung cấp LLM nào? Giá, giới hạn tốc độ, chế độ JSON, tiếng Việt?	Spike S5 với cùng bộ 20 mẫu	C07, C31, AiClient	BE2, tuần 5
Lưu ảnh ở đâu khi deploy?	Chương 8; spike S7 với nơi lưu thật	C12	BE2, khi web ổn định
Dịch vụ PostgreSQL cụ thể nào (theo nguyên tắc C06)? Giới hạn gói miễn phí?	Spike S11; trang giá của 2–3 dịch vụ	Cấu hình kết nối deploy	BE4, trước tuần 7
Đối tác xem chứng từ chính hãng từ lúc nào?	Thảo luận nhóm, đối chiếu SRS 6.16	C26, quyền xem tệp private	BE4 + nhóm, trước tuần 8
VNPay sandbox đăng ký thế nào? Nhận IPN ở localhost ra sao?	Tài liệu VNPay, spike S1	Nạp tiền tuần 5	BE1
Chat dùng polling hay WebSocket?	Spike S9	API chat, FE	BE3 + FE
Deploy backend ở đâu (VPS hay dịch vụ chạy container)? Có HTTPS không?	So sánh 2 lựa chọn; kiểm RAM tối thiểu cho Spring Boot	Deploy thử tuần 7	BE1
Gmail App Password giới hạn bao nhiêu email/ngày?	Đọc giới hạn Gmail SMTP	Demo OTP	BE1

8. Phương án lưu hình ảnh khi deploy (C12)

Ảnh là phần dữ liệu lớn nhất của ShareLoop: mỗi bài 3–8 ảnh, cộng ảnh lỗi, chứng từ hàng chính hãng (private), ảnh chat và minh chứng giao nhận (private). Nhờ interface **StorageService** và việc cơ sở dữ liệu chỉ lưu **storage_key**, nhóm có thể đổi nơi lưu bất cứ lúc nào mà không sửa nghiệp vụ. Vì vậy nhóm quyết định **chốt C12 sau khi web chạy ổn định**: trong lúc code chỉ dùng bản Local, chương này là căn cứ để chọn khi tới lúc deploy.

8.1. Ước lượng dung lượng

Giả định	Giá trị
Ảnh sau khi nén ở backend (cạnh dài tối đa 1600 px, JPEG/WebP chất lượng ~80)	khoảng 200–300 KB/ảnh
Số ảnh trung bình mỗi bài	5 ảnh → khoảng 1,0–1,5 MB/bài
Quy mô demo (khoảng 200 bài, 100 giao dịch có minh chứng)	dưới 0,5 GB
Quy mô thử nghiệm thật (khoảng 5.000 bài)	khoảng 5–8 GB

Kết luận: với quy mô đề án, **mọi gói miễn phí dưới đây đều đủ dung lượng**. Tiêu chí quyết định là độ ổn định khi demo, băng thông, độ khó tích hợp và việc có hỗ trợ tệp private hay không.

8.2. So sánh các phương án

Phương án	Chi phí / gói miễn phí	Ưu điểm	Nhược điểm và rủi ro
A. Ổ đĩa của VPS (LocalStorageService , Nginx phục vụ ảnh)	Không thêm chi phí; dùng dung lượng VPS	Đơn giản nhất; không cần tài khoản dịch vụ khác; hợp khi demo	Mất ảnh nếu VPS hỏng hoặc đổi máy; tốn băng thông VPS; khó mở rộng; phải tự làm URL có hạn cho tệp private
B. Cloudflare R2 (tương thích S3)	Miễn phí 10 GB-tháng, 1 triệu thao tác loại A, 10 triệu thao tác loại B mỗi tháng; vượt thì \$0,015/GB-tháng; không tính phí băng thông tải xuống	Dùng thư viện S3 chuẩn; URL có hạn cho tệp private; gói miễn phí không hết hạn; không lo hoá đơn băng thông	Có thể phải thêm phương thức thanh toán khi kích hoạt (kiểm tra lúc đăng ký); cần cấu hình tên miền hoặc URL công khai cho bucket
C. AWS S3 (+ CloudFront)	Tài khoản mới từ 15/07/2025: nhận 100 USD tín dụng, kiểm thêm tối đa 100 USD; gói miễn phí kéo dài 6 tháng hoặc tới khi hết tín dụng	Chuẩn công nghiệp, tài liệu nhiều; hợp nếu nhóm deploy cả backend trên AWS	Sau 6 tháng phải chuyển gói trả phí; có phí băng thông; cấu hình IAM, chính sách bucket phức tạp hơn; phải đặt cảnh báo ngân sách
D. Supabase Storage	Gói miễn phí: 1 GB lưu trữ, 5 GB băng thông, tệp tối đa 50 MB	Tiện nếu dùng Supabase làm PostgreSQL; có URL có hạn	Dự án miễn phí bị tạm dừng sau 1 tuần không hoạt động — rủi ro ngay trước buổi demo; 1 GB khá nhỏ
E. Cloudinary	Gói miễn phí 25 credit/tháng; 1 credit = 1 GB lưu trữ, hoặc 1 GB băng thông, hoặc 1.000 lượt biến đổi ảnh	Tự nén, đổi cỡ, cắt ảnh qua URL; CDN sẵn	Không dùng API S3 → phải viết bản cài đặt StorageService riêng; tệp private phức tạp hơn; credit dùng chung cho lưu trữ và băng thông

Phương án	Chi phí / gói miễn phí	Ưu điểm	Nhược điểm và rủi ro
F. MinIO tự chạy trên VPS	Không thêm chi phí	API S3 giống hệt R2/S3 → cùng một mã nguồn; tốt cho môi trường dev	Vẫn nằm trên ổ VPS nên có cùng rủi ro mất dữ liệu như A; tốn RAM của VPS
G. Lưu ảnh trong PostgreSQL (bytea)	Không thêm chi phí	Một nơi lưu duy nhất	Không khuyến nghị: database phình to, sao lưu chậm, vượt nhanh giới hạn dung lượng của gói database miễn phí, mọi lượt xem ảnh đều đi qua backend

Khuyến nghị — chốt khi web chạy ổn định

- **Từ nay tới khi web ổn định:** chỉ dùng **LocalStorageService**. Bản S3 chạy thử với MinIO trong Docker Compose khi rảnh (spike S7), không chặn tính năng nào.
- **Thời điểm chốt:** khi luồng đăng bài → giao dịch → hoàn tất chạy ổn trên **develop** (dự kiến cuối tuần 6), trước deploy thử tuần 7.
- **Ứng viên hàng đầu: Cloudflare R2** — hai bucket **shareloop-public** (ảnh bài, ảnh đại diện) và **shareloop-private** (chứng từ, minh chứng, ảnh chat), URL có hạn 10 phút cho bucket private.
- Chọn **AWS S3** thay R2 chỉ khi nhóm deploy toàn bộ trên AWS và theo dõi được hạn 6 tháng của gói miễn phí.
- **Dự phòng khi demo:** **STORAGE_TYPE=local** trên VPS nếu dịch vụ ngoài có sự cố; mã nguồn không đổi.
- Không chọn Supabase Storage cho ảnh vì rủi ro dự án bị tạm dừng; không lưu ảnh trong database.

8.3. Thiết kế StorageService

```
public interface StorageService {
    StoredObject put(String key, InputStream data, long size, String contentType,
        Visibility v);
    String publicUrl(String key); // ảnh công khai
    String signedUrl(String key, Duration ttl); // tệp private, URL có hạn
    void delete(String key);
}
// Hai bản cài đặt, chọn bằng câu hình app.storage.type = local | s3
// S3StorageService dùng AWS SDK v2; với R2: endpointOverride =
https://<account>.r2.cloudflarestorage.com, region = "auto"
```

Quy ước	Nội dung
Đặt tên khoá	public/{userId}/{uuid}.webp , avatars/{userId}/{uuid}.webp (public); private/{userId}/{uuid} , evidence/{requestId}/{uuid} , chat/{requestId}/{uuid} (private)
Luồng tải lên	FE gửi multipart lên backend → kiểm tra loại tệp bằng nội dung (không tin đuôi tệp), ≤ 5 MB → xoá EXIF (ảnh điện thoại có toạ độ GPS nhà người dùng) → đổi cỡ, nén → lưu → tạo media_files → trả mediaId
Quyền xem	Ảnh bài: URL công khai. Chứng từ: chỉ Admin và đối tác (thời điểm theo C26, đề xuất từ lúc Reserved). Minh chứng: Admin và hai bên giao dịch. Backend kiểm quyền rồi mới trả URL có hạn.

Quy ước	Nội dung
Dọn rác	Job mỗi giờ: tệp còn status UPLOADED (chưa gắn vào bài, tin nhắn hay giao dịch) quá 24 giờ thì đặt <code>is_deleted = true</code> và xoá khỏi nơi lưu (ERD mục 6).
Cấu hình	Biến môi trường: <code>STORAGE_TYPE</code> , <code>S3_ENDPOINT</code> , <code>S3_BUCKET_PUBLIC</code> , <code>S3_BUCKET_PRIVATE</code> , <code>S3_ACCESS_KEY</code> , <code>S3_SECRET_KEY</code> , <code>PUBLIC_BASE_URL</code> . Spring: <code>spring.servlet.multipart.max-file-size=5MB</code> .
Bảo mật	Bucket private tắt truy cập công khai; khoá truy cập chỉ có quyền trên đúng hai bucket; cấu hình CORS chỉ cho tên miền FE; không commit khoá.

8.4. Các bước triển khai lưu ảnh

- Tuần 4: `MediaService` + `POST /media` dùng bản Local.
- Tuần 5–6: khi rảnh, chạy spike S7 với MinIO để bản S3 sẵn sàng.
- Khi web ổn định (dự kiến cuối tuần 6): tạo tài khoản R2 (hoặc phương án được chọn) và hai bucket, chạy lại spike với nơi lưu thật; **chốt C12** và ghi vào `docs/decisions/`.
- Tuần 7 (deploy thử): đổi biến môi trường sang S3/R2; kiểm tra tải lên, xem ảnh, URL có hạn hết hiệu lực sau 10 phút.
- Tuần 9, trước buổi demo: mở thử trang có ảnh; giữ sẵn cấu hình dự phòng `STORAGE_TYPE=local`.

9. Lịch làm việc từ tuần 4 đến tuần 9

9.1. Tuần 4 theo từng ngày

Ngày	Việc chính	Kết quả cuối ngày
T2 28/9	Chốt quyết định và ERD (xong buổi sáng). Mọi người cài JDK 21, IDE, Docker	Biên bản; máy mọi người sẵn sàng
T3 29/9	Bổ sung cột chuẩn theo góp ý giảng viên (C34); chốt Spring Boot 4.1.1. Tối: BE1 dựng nền repo theo Hướng dẫn BE1 (B1–B3). Mỗi BE viết API contract module mình	Tài liệu cập nhật 29/09; repo có luật chặn ghi công AI và AGENTS.md
T4 30/9	BE1 tạo project Spring Boot 4.1.1 theo cây thư mục chương 4, Docker Compose, CI + Spotless, CODEOWNERS, README (PR #1); V1__init.sql (PR #2). BE4 viết V2, V4 ; BE2 viết V3 . FE review contract; tạo backlog sprint tuần 4	Repo chạy được, CI xanh; migration chạy sạch
T5 1/10	BE1: lớp nền (BaseEntity , lỗi thống nhất, JWT, Swagger), endpoint mẫu có test. Mỗi chủ service cửa ngõ merge chữ ký hàm (mục 4.7). Mọi người bắt đầu code sprint tuần 4 trên nhánh riêng	Checklist chương 10 tích đủ các mục bắt buộc
T6 2/10 – T7 3/10	Code sprint tuần 4 (đăng ký, OTP email, đăng nhập, tài ảnh, tạo bài, gửi yêu cầu, cấu hình); spike S2, S3, S4, S6, S8 xen kẽ	Mỗi người có ít nhất một PR được merge
CN 4/10	Demo nội bộ trên deve lop; lập kế hoạch tuần 5	Mốc tuần 4: đăng ký → đăng nhập → tạo bài chạy đầu–cuối

9.2. Các tuần tiếp theo

Tuần	Trọng tâm	Spike / việc hạ tầng đi kèm	Mốc
5 (5/10 – 11/10)	Ví, nạp tiền, phí đăng bài; kiểm duyệt tầng 1 và tầng 3; chọn người, mở chat	S1 VNPAY (đầu tuần), S5 LLM → chốt C07, C31; S9 chat; S11 PostgreSQL	Mốc 1: đăng bài có phí + duyệt
6 (12/10 – 18/10)	Chat lọc liên hệ, chốt lịch, trao liên lạc, giao nhận; tìm kiếm; AI sàng lọc bài; sửa bài (phí 5 Credit), gia hạn; khiếu nại, uy tín	S10 job nền; cuối tuần: web ổn định → S7 với nơi lưu thật → chốt C12	Mốc 2: giao dịch tới Completed — điểm kiểm tra cắt giảm
7 (19/10 – 25/10)	AI trợ lý (2 Credit/lượt), AI chat tầng 2; báo cáo tài chính; đẩy bài; thẻ bài trong chat	Deploy thử lần 1: VPS + PostgreSQL managed + nơi lưu ảnh đã chốt	Mốc 3: xong Must, chạy online
8 (26/10 – 1/11)	Kiểm thử tích hợp, đo chất lượng bộ lọc, Should theo thứ tự ưu tiên; chốt C26 trước khi làm xác thực chính hãng	Deploy bản ổn định	Đóng băng tính năng
9 (2/11 – 8/11)	Sửa lỗi, giao diện, tài liệu, kịch bản và tập duyệt demo	Chuẩn bị chế độ dự phòng (lưu ảnh local, cổng thanh toán giả lập)	Nộp và bảo vệ

Lịch giả định mỗi người dành khoảng 3 giờ mỗi ngày cho dự án. Chi tiết việc của từng người mỗi tuần nằm ở SRS chương 16 và bảng mục 4.5.

10. Checklist sẵn sàng code

Tích từng ô trong buổi họp tối thứ Năm 1/10. Hai mục có đánh dấu (*) được phép xong muộn hơn, trong tuần 4–5.

	Mục	Người xác nhận
☒	Biên bản các quyết định nhóm 1 và ERD 17 bảng; SRS đã sửa theo (xong 28/9)	BE1
☐	Cả 4 BE đã cài JDK 21, IDE, Docker; <code>./mvnw verify</code> chạy xanh trên máy mình	Từng người
☐	Cây thư mục đủ 18 module theo chương 4; <code>package-info.java</code> ghi người sở hữu	BE1
☐	CODEOWNERS, <code>.editorconfig</code> , <code>.gitattributes</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code> , <code>.gemini/settings.json</code> đã có trong repo	BE1
☐	<code>V1__init.sql</code> và <code>V2–V4</code> chạy sạch trên database trống	BE1, BE2, BE4
☐	Sơ đồ use case và screen flow cập nhật theo các mức phí đã chốt	BE1, FE
☐	API contract tuần 4–5 + danh sách mã lỗi theo module	BE + FE
☐	Service cửa ngõ (mục 4.7) đã có chữ ký hàm và được merge	Chủ từng service
☐	Endpoint mẫu có test chạy xanh; Swagger UI mở được	BE1
☐	CI xanh (Spotless + test); nhánh <code>develop</code> , <code>main</code> đã bật bảo vệ và bắt buộc review code owner	BE1
☐	README có hướng dẫn chạy và mục Quy ước	BE1
☐	Bí mật (API key, mật khẩu SMTP, JWT secret) nằm trong biến môi trường, không có trong repo	BE1
☐	Backlog sprint tuần 4 trên GitHub Projects, mỗi issue có người nhận và nhãn module	BE1
☐	(*) Báo cáo 11 spike (xong / chưa xong + ghi chú)	Người phụ trách spike
☐	(*) Bản Local của StorageService chạy thật; nơi lưu khi deploy chốt sau (C12)	BE2

11. Những lỗi hay gặp khi bắt đầu và cách tránh

Lỗi	Hậu quả	Cách tránh
Để <code>ddl-auto=update</code> tự tạo bảng	Schema mỗi máy một kiểu; lên production vỡ	Luôn <code>validate</code> + Flyway ngay từ đầu
Viết business rule trong controller	Trùng lặp, khó test, lệch SRS	Controller chỉ gọi service; review từ chối PR vi phạm
Trả entity ra API	Lộ số điện thoại, vòng lặp JSON do quan hệ hai chiều	Luôn dùng DTO <code>record</code>
Cộng trừ Credit ở nhiều nơi	Sai số dư, không truy vết được	Chỉ <code>CreditService</code> đụng tới tiền, có khoá dòng và ghi ledger
Inject repository của module khác "cho nhanh"	Hai người cùng sửa một bảng, luật nghiệp vụ bị bỏ qua	Gọi service cửa ngõ; bật <code>ModuleBoundaryTest</code> nếu tái phạm
Mọi người cùng sửa <code>User.java</code>	Xung đột mỗi ngày	Entity lát cắt theo nhóm cột (mục 4.8)
Hai người tạo migration cùng số <code>V5</code>	Flyway báo lỗi, database dev hỏng	Version theo thời gian, <code>out-of-order</code> ở dev (mục 4.9)
Mỗi IDE định dạng một kiểu, file CRLF lẫn LF	PR phình to, xung đột giả, <code>mvnw</code> lỗi trên CI	<code>.editorconfig</code> , <code>.gitattributes</code> , Spotless trong CI
Để trợ lý AI sửa lan ra module khác hoặc file nền	PR khó review, phá mã của người khác	<code>AGENTS.md</code> ghi rõ phạm vi; agent hỏi trước khi sửa; CODEOWNERS chặn
Xoá cứng bản ghi hoặc tự gán <code>created_by, updated_at</code>	Mất dữ liệu đối soát; cột chuẩn sai người, sai giờ	Gọi <code>markDeleted()</code> ; để JPA Auditing và trigger điền cột chuẩn (ERD 7.11)
Làm theo hướng dẫn Spring Boot 3 (<code>spring-boot-starter-web</code> , <code>@MockBean</code> , <code>com.fasterxml.jackson.databind</code>)	Lỗi biên dịch, thiếu cấu hình tự động	Đọc hộp lưu ý mục 3.5.1; kiểm tra import và tên starter trước
Test bằng H2 thay PostgreSQL	Test xanh nhưng chạy thật lỗi (JSONB, unaccent, cú pháp khác)	Testcontainers
Để PR quá lớn, merge cuối tuần	Xung đột nhiều, review qua loa	PR nhỏ, merge hằng ngày vào <code>develop</code>
Commit API key lên GitHub	Mất tiền, key bị thu hồi	Biến môi trường, <code>.env</code> trong <code>.gitignore</code> ; bật secret scanning của GitHub
Chờ FE xong mới làm BE hoặc ngược lại	Dồn việc cuối kỳ	Contract trước, mock sau, làm song song
Gọi AI đồng bộ khi người dùng bấm gửi bài	Thao tác chậm vài giây, treo khi nhà cung cấp lỗi	Chạy bất đồng bộ sau commit; có timeout và nhãn "AI chưa kiểm"

Nguồn tham khảo

- Cloudflare R2 — trang sản phẩm và bảng giá: <https://www.cloudflare.com/products/r2/>
- AWS — thông báo gói Free Tier mới (tín dụng 100–200 USD, gói miễn phí 6 tháng), 07/2025: <https://aws.amazon.com/about-aws/whats-new/2025/07/aws-free-tier-credits-month-free-plan/>
- Supabase — bảng giá: <https://supabase.com/pricing>
- Cloudinary — bảng giá: <https://cloudinary.com/pricing>
- HeroDevs — các nhánh Spring Boot, ngày hết hỗ trợ (3.5 hết hỗ trợ OSS 30/06/2026): <https://www.herodevs.com/blog-posts/spring-boot-versions-eol-dates-and-latest-releases-april-2026>
- springdoc-openapi — dòng 2.x cho Spring Boot 3, dòng 3.x cho Spring Boot 4: <https://github.com/springdoc/springdoc-openapi>
- Spring Boot 4.1.1 (20/08/2026): <https://spring.io/blog/2026/08/20/spring-boot-4-1-1-available-now/>
- Spring Boot 4.0 Migration Guide (đổi tên starter, starter Flyway, @MockitoBean, Jackson 3): <https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide>
- Testcontainers 2 — artifact testcontainers-postgresql: <https://mvnrepository.com/artifact/org.testcontainers/testcontainers-postgresql>
- Google Antigravity — Rules (đọc AGENTS.md, GEMINI.md, .agents/rules): <https://antigravity.google/docs/rules/>
- So sánh CLAUDE.md, AGENTS.md, GEMINI.md và cách trở về một file chung: <https://inventivehq.com/blog/claude-md-vs-agents-md-vs-gemini-md>
- Giá, gói miễn phí và phiên bản công cụ thay đổi thường xuyên; kiểm tra lại khi đăng ký hoặc khi tạo project (số liệu lưu ảnh tra ngày 26/09/2026, phiên bản tra ngày 29/09/2026).